Advanced Redux

Easy Guide

Contents

Chapter 1

Redux & Side Effects (and Asynchronous Code)

1.1 Understanding Side Effects

A side effect is anything that happens outside your reducer that changes the world. Common side effects:

- Fetching data from an API
- Making HTTP requests
- Writing to a file
- Sending an email
- Logging to console
- Setting timers
- Local storage operations

1.2 Why Side Effects Are Tricky

Reducers must be pure functions. They cannot:

- Make API calls
- Have unpredictable outcomes
- Modify things outside themselves
- Use random numbers
- Depend on current time

So where do side effects go?

1.3 Where to Put Side Effects

Location	Good For
Components (useEffect)	Simple effects
Redux Middleware	Complex effects
Thunks (Action Creators)	Async operations

1.4 The Three Approaches

1.4.1 1. useEffect in Components

Simple and straightforward:

```
useEffect(() => {
  // Side effect code here
}, []);
```

1.4.2 2. Redux Middleware

Advanced pattern for complex logic

1.4.3 3. Thunks

Special Redux functions for async operations

Choice: Start with useEffect, graduate to thunks as needs grow.

Chapter 2

Redux & Async Code

2.1 The Challenge

Reducers are synchronous (instant). But API calls are asynchronous (take time).

2.2 Timeline Problem

```
Dispatch action
  ↓ (Reducer runs immediately)
State updates instantly
  ↓ (But API response not ready!)
Component renders with wrong data
  ↓ (After 2 seconds)
API response arrives
  ↓ (But reducer already ran!)
```

2.3 The Solution: Dispatch Multiple Actions

Instead of one action, dispatch multiple:

1. **LOADING** - API call started
2. **SUCCESS** - Data arrived, update state
3. **ERROR** - Something went wrong

2.4 Example

```
// Action 1: Request starts
dispatch({ type: 'FETCH_START' });

// Wait for API response...
try {
  const data = await fetch('/api/users');
```

```
  // Action 2: Success
  dispatch({ type: 'FETCH_SUCCESS', payload: data });
} catch (error) {

  // Action 3: Error
  dispatch({ type: 'FETCH_ERROR', payload: error });
}
```

2.5 Handling These States

```
const userSlice = createSlice({
  name: 'users',
  initialState: {
    data: [],
    loading: false,
    error: null
  },
  reducers: {
    fetchStart: (state) => {
      state.loading = true;
      state.error = null;
    },
    fetchSuccess: (state, action) => {
      state.data = action.payload;
      state.loading = false;
    },
    fetchError: (state, action) => {
      state.error = action.payload;
      state.loading = false;
    }
  }
});
```

Pattern: Track loading, success, and error states separately!

Chapter 3

Using useEffect with Redux

3.1 Combining useEffect and Redux

`useEffect` is the perfect place to fetch data and dispatch Redux actions:

```
import { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchUsers } from '../slices/userSlice';

function Users() {
  const dispatch = useDispatch();
  const { data, loading, error } = useSelector(state => state.users);

  useEffect(() => {
    // Fetch when component mounts
    dispatch(fetchUsers());
  }, [dispatch]);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

export default Users;
```

3.2 useEffect Dependency Array

```
// Runs once on mount
```

```
useEffect(() => { ... }, []);

// Runs when dispatch changes
useEffect(() => { ... }, [dispatch]);

// Runs when userId changes
useEffect(() => { ... }, [userId]);

// Runs on every render (AVOID!)
useEffect(() => { ... });
```

3.3 Fetching Based on Props

```
function UserDetail({ userId }) {
  const dispatch = useDispatch();
  const user = useSelector(state => state.users.currentUser);

  useEffect(() => {
    // Re-fetch when userId changes
    dispatch(fetchUserDetail(userId));
  }, [userId, dispatch]);

  return <div>{user?.name}</div>;
}
```

3.4 Cleanup in useEffect

```
useEffect(() => {
  let isMounted = true;

  const fetchData = async () => {
    const data = await fetch('/api/data');
    if (isMounted) {
      dispatch(setData(data));
    }
  };

  fetchData();

  // Cleanup function
  return () => {
    isMounted = false; // Prevent setting state if unmounted
  };
}, [dispatch]);
```

Best Practice: Always clean up to prevent memory leaks!

Chapter 4

A Problem with useEffect()

4.1 The Infinite Loop Problem

The most common mistake:

```
// BAD: Infinite loop!
useEffect(() => {
  dispatch(fetchData());
}, [dispatch]); // dispatch changes every render
```

Why? Because `dispatch` is recreated every render, causing useEffect to run infinitely.

4.2 Solutions

4.2.1 Solution 1: Empty Dependency Array

```
// GOOD: Runs only on mount
useEffect(() => {
  dispatch(fetchData());
}, []);
```

Use this when you only need to fetch once.

4.2.2 Solution 2: Include dispatch

Actually, React optimizes this. Dispatch is stable:

```
useEffect(() => {
  dispatch(fetchData());
}, [dispatch]); // Safe in Redux apps
```

4.2.3 Solution 3: Fetch Function Only Once

```
useEffect(() => {
  const fetchAndDispatch = async () => {
    const data = await fetch('/api/data');
    dispatch(setData(data));
```

```
  };

  fetchAndDispatch();
}, []); // Empty array - runs once
```

4.3 Other Common Problems

4.3.1 Fetching Same Data Twice

```
// BAD: Fetches twice in development (StrictMode)
useEffect(() => {
  dispatch(fetchData());
}, [dispatch]);

// GOOD: Check if data already exists
useEffect(() => {
  if (data.length === 0) {
    dispatch(fetchData());
  }
}, [dispatch, data]);
```

4.3.2 Not Tracking Loading State

```
// BAD: No feedback to user
useEffect(() => {
  dispatch(fetchData());
}, []);

// GOOD: Show loading state
const { loading, error } = useSelector(state => state.data);

if (loading) return <Spinner />;
if (error) return <Error />;
```

Common Trap: Forgetting to handle loading and error states!

Chapter 5

Handling Http States & Feedback with Redux

5.1 The Three HTTP States

Every API call has three possible outcomes:

1. **Loading** - Request in progress
2. **Success** - Data received
3. **Error** - Something went wrong

5.2 Tracking These States

```
const dataSlice = createSlice({
  name: 'data',
  initialState: {
    items: [],
    status: 'idle', // 'idle', 'loading', 'success', 'error'
    error: null
  },
  reducers: {
    fetchStart: (state) => {
      state.status = 'loading';
      state.error = null;
    },
    fetchSuccess: (state, action) => {
      state.status = 'success';
      state.items = action.payload;
    },
    fetchError: (state, action) => {
      state.status = 'error';
      state.error = action.payload;
    }
  }
```

```
});
```

5.3 Providing Feedback to Users

5.3.1 Show Loading Spinner

```
function DataList() {
  const { items, status, error } = useSelector(state => state.data);

  if (status === 'loading') {
    return <Spinner />;
  }

  if (status === 'error') {
    return <ErrorMessage error={error} />;
  }

  if (status === 'success') {
    return (
      <ul>
        {items.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    );
  }

  return null;
}
```

5.3.2 Show Error Message

```
if (status === 'error') {
  return (
    <div className="error">
      <p>Failed to load data</p>
      <p className="error-detail">{error}</p>
      <button onClick={() => dispatch(retryFetch())}>
        Try Again
      </button>
    </div>
  );
}
```

5.3.3 Show Success Message

```
if (status === 'success' && items.length === 0) {
  return <p>No items found</p>;
```

```
}

if (status === 'success' && items.length > 0) {
  return <p>Loaded {items.length} items successfully</p>;
}
```

5.4 Timeout Handling

```
const fetchWithTimeout = async (url, timeout = 5000) => {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeout);

  try {
    const response = await fetch(url,
      { signal: controller.signal }
    );
    clearTimeout(timeoutId);
    return response.json();
  } catch (error) {
    if (error.name === 'AbortError') {
      throw new Error('Request timeout');
    }
    throw error;
  }
};
```

UX Rule: Always show loading, error, and success states!

Chapter 6

Using an Action Creator Thunk

6.1 What is a Thunk?

A thunk is a special function that handles async operations in Redux. It lets you write logic that interacts with the Redux store.

6.2 Without Thunk

```
// In component
useEffect(() => {
  const fetchData = async () => {
    dispatch(fetchStart());
    try {
      const response = await fetch('/api/data');
      const data = await response.json();
      dispatch(fetchSuccess(data));
    } catch (error) {
      dispatch(fetchError(error.message));
    }
  };

  fetchData();
}, []);
```

Lots of boilerplate in the component!

6.3 With Thunk

```
// In slice
export const fetchData = createAsyncThunk(
  'data/fetchData',
  async (_, { rejectWithValue }) => {
    try {
      const response = await fetch('/api/data');
      const data = await response.json();
```

```
      return data;
    } catch (error) {
      return rejectWithValue(error.message);
    }
  }
);

// Handle the thunk in reducers
extraReducers: (builder) => {
  builder
    .addCase(fetchData.pending, (state) => {
      state.status = 'loading';
    })
    .addCase(fetchData.fulfilled, (state, action) => {
      state.status = 'success';
      state.items = action.payload;
    })
    .addCase(fetchData.rejected, (state, action) => {
      state.status = 'error';
      state.error = action.payload;
    });
}
```

6.4 In Component

```
function DataList() {
  const dispatch = useDispatch();
  const { items, status } = useSelector(state => state.data);

  useEffect(() => {
    dispatch(fetchData());
  }, [dispatch]);

  if (status === 'loading') return <Spinner />;
  if (status === 'error') return <Error />;

  return <List items={items} />;
}
```

Much cleaner! All async logic is in one place.

6.5 Benefits of Thunks

- Cleaner components
- Centralized async logic
- Consistent error handling

- Easy to test
- Redux DevTools integration

Modern Approach: Use createAsyncThunk for most async operations!

Chapter 7

Exploring Fetching Data

7.1 Complete Data Fetching Pattern

7.1.1 1. Create the Thunk

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';

export const fetchUsers = createAsyncThunk(
  'users/fetchUsers',
  async (_, { rejectWithValue }) => {
    try {
      const response = await fetch('/api/users');
      if (!response.ok) {
        throw new Error('Failed to fetch');
      }
      return await response.json();
    } catch (error) {
      return rejectWithValue(error.message);
    }
  }
);
```

7.1.2 2. Handle in Slice

```
const userSlice = createSlice({
  name: 'users',
  initialState: {
    list: [],
    status: 'idle',
    error: null
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => {
        state.status = 'loading';
```

```
        state.error = null;
      })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = 'success';
        state.list = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.status = 'error';
        state.error = action.payload;
      });
  }
});

export default userSlice.reducer;
```

7.1.3 3. Use in Component

```
function UserList() {
  const dispatch = useDispatch();
  const { list, status, error } = useSelector(state => state.users);

  useEffect(() => {
    dispatch(fetchUsers());
  }, [dispatch]);

  return (
    <div>
      {status === 'loading' && <p>Loading users...</p>}
      {status === 'error' && <p>Error: {error}</p>}
      {status === 'success' && (
        <ul>
          {list.map(user => (
            <li key={user.id}>{user.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

7.2 Fetching with Parameters

```
export const fetchUserById = createAsyncThunk(
  'users/fetchUserById',
  async (userId, { rejectWithValue }) => {
    try {
      const response = await fetch(`/api/users/${userId}`);
      return await response.json();
```

```
    } catch (error) {
      return rejectWithValue(error.message);
    }
  }
);

// In component
dispatch(fetchUserById(123)); // Pass the ID
```

7.3 POST Request Example

```
export const createUser = createAsyncThunk(
  'users/createUser',
  async (userData, { rejectWithValue }) => {
    try {
      const response = await fetch('/api/users', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(userData)
      });
      return await response.json();
    } catch (error) {
      return rejectWithValue(error.message);
    }
  }
);
```

7.4 Handling Errors Better

```
export const fetchData = createAsyncThunk(
  'data/fetch',
  async (url, { rejectWithValue }) => {
    try {
      const response = await fetch(url);

      if (!response.ok) {
        const errorData = await response.json();
        throw new Error(errorData.message);
      }

      return await response.json();
    } catch (error) {
      return rejectWithValue({
        message: error.message,
        timestamp: new Date().toISOString()
      });
    }
```

```
  }
);
```

Pattern: Thunk for fetching → handle states → display in component!

Chapter 8

Exploring the Redux DevTools

8.1 What are Redux DevTools?

Redux DevTools is a browser extension that helps you debug Redux applications. It shows:

- Every action dispatched
- State before and after each action
- Time-travel debugging
- Action history
- State diffs

8.2 Installing Redux DevTools

8.2.1 Step 1: Install Browser Extension

Search for "Redux DevTools" in your browser's extension store:

- Chrome: Redux DevTools
- Firefox: Redux DevTools

8.2.2 Step 2: Your Code Already Supports It!

Redux Toolkit's `configureStore()` includes DevTools support by default:

```
const store = configureStore({
  reducer: {
    counter: counterReducer
  }
  // DevTools automatically enabled!
});
```

No extra setup needed!

8.3 Using Redux DevTools

8.3.1 1. Open DevTools

Click the Redux DevTools icon in your browser toolbar.

8.3.2 2. See Action History

Every action appears in the DevTools:

```
Action: @@INIT
Action: counter/increment
Action: counter/increment
Action: counter/decrement
```

8.3.3 3. Inspect State

Click any action to see:

- State before the action
- State after the action
- What changed (diff)

8.3.4 4. Time-Travel Debugging

Click any previous action to jump back in time:

```
Counter: 5
↓ (Click an earlier action)
Counter: 2
↓ (The app shows that state immediately!)
```

This doesn't actually go back in time in your app, but you can see what the state was.

8.4 DevTools Features

8.4.1 Dispatch Tab

Manually dispatch actions for testing:

```
{
  "type": "counter/increment",
  "payload": 5
}
```

Click "Dispatch" to test without user interaction.

8.4.2 Action Diff Tab

See exactly what changed:

```
State before: { counter: 5 }
State after:  { counter: 6 }

Changed: counter: 5 → 6
```

8.4.3 State Tab

View entire Redux state in JSON format:

```
{
  "counter": { "value": 5 },
  "user": { "name": "John" },
  "items": { "list": [...] }
}
```

8.5 Common Debugging Scenarios

8.5.1 Finding Where State Changed

1. Open DevTools
2. Look at action history
3. Click the action that changed state
4. See the difference
5. Find the reducer that handled it

8.5.2 Testing Edge Cases

Use the Dispatch tab to:

- Manually dispatch actions
- Test error scenarios
- Verify state updates
- Test async flows

8.5.3 Performance Debugging

DevTools shows:

- Which actions take long
- Frequency of actions
- State size

8.6 DevTools Configuration

To customize DevTools (usually not needed):

```
import { configureStore } from '@reduxjs/toolkit';

const store = configureStore({
  reducer: {
    // your reducers
  },
  devTools: {
    // Optional configuration
    trace: true,
    traceLimit: 25
  }
});
```

8.6.1 Disable in Production

```
devTools: process.env.NODE_ENV !== 'production'
```

Superpower: Redux DevTools makes debugging Redux incredibly easy!

8.7 Tips & Tricks

8.7.1 Jump to Specific Action

Click any action in history to see state at that moment.

8.7.2 Export/Import State

Export Redux state for sharing bugs:

- Click “Export” in DevTools
- Send the JSON to teammates
- They can import to reproduce issue

8.7.3 Lock Replay

Lock the current state and replay actions:

- Click “Lock” button
- State won’t change from UI
- Good for testing specific scenarios

Pro Tip: Redux DevTools is your best friend for debugging!

Quick Summary

Key Concepts

1. **Side Effects** - Things that happen outside reducers
2. **Async Code** - Operations that take time
3. **HTTP States** - Loading, success, error
4. **Thunks** - Redux functions for async logic
5. **DevTools** - Debugging tool for Redux

The Async Pattern

```
useEffect dispatches thunk
    ↓
Thunk starts (PENDING)
    ↓
API call happens
    ↓
Success: dispatch FULFILLED + data
    ↓
Or Error: dispatch REJECTED + error
    ↓
Reducer updates state
    ↓
Component re-renders with new state
```

Complete Example

```
// 1. Create thunk
export const fetchData = createAsyncThunk('data/fetch', async () => {
  const response = await fetch('/api/data');
  return response.json();
});

// 2. Handle in slice
extraReducers: (builder) => {
```

```
  builder
    .addCase(fetchData.pending, state => { state.loading = true; })
    .addCase(fetchData.fulfilled, (state, action) => {
      state.loading = false;
      state.data = action.payload;
    })
    .addCase(fetchData.rejected, (state, action) => {
      state.loading = false;
      state.error = action.payload;
    });
}

// 3. Use in component
useEffect(() => {
  dispatch(fetchData());
}, [dispatch]);
```

Best Practices

1. Track three states: loading, success, error
2. Use createAsyncThunk for async operations
3. Handle errors gracefully
4. Show loading spinners to users
5. Use Redux DevTools for debugging
6. Keep async logic in thunks, not components
7. Always clean up useEffect
8. Validate API responses

Common Mistakes to Avoid

- Fetching in reducers (not allowed!)
- Forgetting loading states
- Not handling errors
- Infinite useEffect loops
- Ignoring Redux DevTools
- Mutating state directly

When to Use Each Tool

Tool	When to Use
useEffect	Simple one-time fetches
Thunks	Complex async logic
Middleware	Advanced patterns
DevTools	All the time for debugging!

Debugging Checklist

1. Open Redux DevTools
2. Look at action history
3. Check state before/after each action
4. Look for unexpected actions
5. Check action payloads
6. Verify error messages
7. Use time-travel to test scenarios

Master async Redux and you'll handle any complex app!